

2026 年 3 月 18 日架构师网络课程纪要

一、本次课程总结

【课堂主题】

《软件工程》

【重点内容】

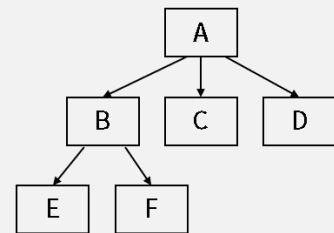
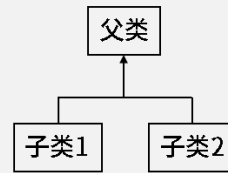




面向对象设计原则



- ★ 单一职责原则：设计目的单一的类
- ★ 开放-封闭原则：对扩展开放，对修改封闭
- ★ 李氏（Liskov）替换原则：子类可以替换父类
- ★ 依赖倒置原则：要依赖于抽象，而不是具体实现；针对接口编程，不要针对实现编程
- ★ 接口隔离原则：使用多个专门的接口比使用单一的总接口要好
- ★ 组合重用原则：要尽量使用组合，而不是继承关系达到重用目的
- ★ 迪米特（Demeter）原则（最少知识原则）：一个对象应当对其他对象有尽可能少的了解



软件测试类型（多种分类方式）



【以测试过程中程序执行状态为依据可分类】

动态测试【计算机运行程序】

静态测试【人工监测和计算机辅助分析】

分为3个步骤：构造测试实例、执行程序以及分析结果。

桌前检查

代码审查

代码走查

都是做的【静态分析】

控制流分析：是否存在没有使用的语句/无法达到的语句/调用并不存在的子程序。

数据流分析：引用未定义的变量、对以前未使用的变量再次赋值。

接口分析：模块之间接口的一致性、子程序和函数之间的接口一致性、函数形参与实参的数量、顺序、类型的一致性。

表达式分析：括号不配对、数组引用越界、除数为零。



白盒测试与黑盒测试

【以内部结构是否可见分类】

白盒测试【结构测试】：关注内部结构与逻辑。

控制流分析、数据流分析、路径分析、程序变异【错误驱动测试】

【路径覆盖】（最强）

【逻辑覆盖】（由强到弱排列）

修正条件/判定 条件组合 条件/判定覆盖 条件覆盖 判定覆盖 语句覆盖

黑盒测试【功能测试】：关注输入输出及功能。

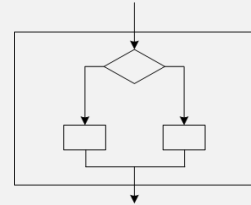
等价类划分：不同等价类，揭示不同问题；有效等价类/无效等价类。

边界值分析： $1 \leq x \leq 10$ ，可取x的值为0、1、10和11作为测试数据

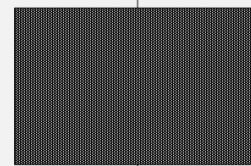
错误推测：依靠测试人员的经验和直觉。

判定表：最适合描述在多个逻辑条件取值的组合所构成的复杂情况下，分别要执行哪些不同的动作。

因果图：根据输入条件与输出结果之间的因果关系来设计测试用例。



输入



功能

输出



软件测试阶段

测试阶段

单元测试：依据【详细设计】，模块测试，模块功能、性能、接口等

集成测试：依据【概要设计】，模块间的接口

系统测试：依据【需求文档】，包括功能测试、性能测试、验收测试、压力测试等



确认测试：依据【需求文档】，验证软件与需求的一致性。
包括内部确认测试、Alpha测试、Beta测试、验收测试



软件测试阶段

其他测试	描述
AB测试	多版本同时使用，利于收集各版本的用户反馈，评估出最好版本。故也算是一种【网页优化方法】。同一时间维度，分别组成成分相同或相似的访客群组。
Web测试	Web系统测试与其他系统测试测试内容基本相同，只是测试重点不同。 Web代码测试包括：源代码规则分析、链接测试、框架测试、表格测试、图形测试等方面。
链接测试	链接测试可分为3个方面： 1、测试所有链接是否按指示的那样确实链接到了该链接的页面。 2、测试所链接的页面是否存在。 3、保证Web应用系统上没有孤立的页面。
表单测试	验证服务器是否能正确保存这些数据，后台运行的程序能否正确解释和使用这些信息。测试提交操作的完整性。
回归测试	测试软件变更之后，变更部分的正确性和对变更需求的符合性。



软件系统测试

➤ 功能测试

➤ 性能测试

- 健壮性测试
- 用户界面测试
- 安全性测试
- 安装与反安装测试

- ✓ 负载测试：各种工作负载下系统的性能
- ✓ 压力测试【测上限】：系统的瓶颈或不能接受的性能点
- ✓ 强度测试【测下限】：系统资源特别低【受限】的情况下运行
- ✓ 容量测试【并发测试】：同时在线的最大用户数
- ✓ 可靠性测试：MTTF之类的参数



软件测试与软件调试



调试是测试之后的活动，测试和调试在目标、方法和思路上都不同。

软件测试	软件调试
目的是找出存在的错误	目的是定位错误并修改程序以修正错误
从一个已知的条件开始，使用预先定义的过程，有预知的结果	从一个未知的条件开始，结束的过程不可预计
测试过程可以事先设计，进度可以事先确定	调试不能描述过程或持续时间



软件维护类型



- ★ **正确性维护【修BUG】**：识别和纠正软件错误/缺陷，测试不可能发现所有错误。
- ★ **适应性维护【应变】**：指使应用软件适应环境变化【外部环境、数据环境】而进行的修改。
- ★ **完善性维护【新需求】**：扩充功能和改善性能而进行的修改。
- ★ **预防性维护【针对未来】**：为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使系统适应各类变化而不会被淘汰。经典实例：**【专用】改【通用】**。

【视频回看地址】

希赛网目前提供了两种课程回看的方式：

方式一：课程回放【课程结束后 20 分钟内可立即回看】

APP 观看路径：我的->网络课堂

PC 观看路径：学习中心->网络课堂

回看注意事项：

- (1) 首先登录希赛网(<https://www.xisaiwang.com/>)，然后使用以上地址回看。
- (2) 回看时如果需要拖放观看,有两种方法进行视频内容定位:方法一是将鼠标移至视频头像位置进行拖放；方法二是直接点击播放界面下方的微缩图，可直接定位到这一页 PPT 的讲解。

方式二：老师端录屏回看【课程结束后 1-2 个工作日发布】

APP 观看路径：我的->视频课程

PC 观看路径：学习中心->视频课程

二、下次课程安排

主题：《案例技巧篇》+《案例特训专题【软件工程篇】》

时间：2026 年 3 月 20 日【周五】19:30 - 21:30

预习要求：

回顾《软件工程》相关知识，案例篇是对软件程基础知识中涉及案例内容的细化讲解。

视频地址为：

<https://wangxiao.xisaiwang.com/nstu/sub/course/courseDetail/courseDetail?courseId=13147>

三、课后作业

1、在面向对象设计中，（ ）可以实现界面控制、外部接口和环境隔离。（ ）作为完成用例业务的责任承担者，协调、控制其他类共同完成用例规定的功能或行为。

- | | | | |
|-------|-------|-------|-------|
| A 实体类 | B 控制类 | C 边界类 | D 交互类 |
| A 实体类 | B 控制类 | C 边界类 | D 交互类 |

2、系统测试由若干个不同的测试类型组成，其中（ ）检查系统能力的最高实际限度，即软件在一些超负荷情况下的运行情况；（ ）主要是检查系统的容错能力。

- | | | | |
|--------|--------|--------|---------|
| A 强度测试 | B 性能测试 | C 恢复测试 | D 可靠性测试 |
| A 强度测试 | B 性能测试 | C 恢复测试 | D 可靠性测试 |

3、（ ）的目的是检查模块之间，以及模块和已集成的软件之间的接口关系，并验证已集成的软件是否符合设计要求。其测试的技术依据是（ ）。

- | | | | |
|-------------|----------|--------|--------|
| A 单元测试 | B 集成测试 | C 系统测试 | D 回归测试 |
| A 软件详细设计说明书 | B 技术开发合同 | | |
| C 软件概要设计文档 | D 软件配置文档 | | |

参考答案及试题解析

1、在面向对象设计中，（ ）可以实现界面控制、外部接口和环境隔离。（ ）作为完成用例业务的责任承担者，协调、控制其他类共同完成用例规定的功能或行为。

- | | | | |
|-------|-------|-------|-------|
| A 实体类 | B 控制类 | C 边界类 | D 交互类 |
| A 实体类 | B 控制类 | C 边界类 | D 交互类 |

参考答案： C B

试题解析：

1. 实体类

实体类映射需求中的每个实体，实体类保存需要存储在永久存储体中的信息，例如，在线教育平台系统可以提取出学员类和课程类，它们都属于实体类。实体类通常都是永久性的，它们所具有的属性和关系是长期需要的，有时甚至在系统的整个生存期都需要。

实体类是对用户来说最有意义的类，通常采用业务领域术语命名，一般来说是一个名词，在用例模型向领域模型的转化中，一个参与者一般对应于实体类。通常可以从 SRS 中的那些与数据库表（需要持久存储）对应的名词着手来找寻实体类。通常情况下，实体类一定有属性，但不一定有操作。

2. 控制类

控制类是用于控制用例工作的类，一般是由动宾结构的短语（“动词+名词”或“名词+动词”）转化来的名词，例如，用例“身份验证”可以对应于一个控制类“身份验证器”，它提供了与身份验证相关的所有操作。控制类用于对一个或几个用例所特有的控制行为进行建模，控制对象（控制类的实例）通常控制其他对象，因此，它们的行为具有协调性。

控制类将用例的特有行为进行封装，控制对象的行为与特定用例的实现密切相关，当系统执行用例的时候，就产生了一个控制对象，控制对象经常在其对应的用例执行完毕后消亡。通常情况下，控制类没有属性，但一定有方法。

3. 边界类

边界类用于封装在用例内、外流动的信息或数据流。边界类位于系统与外界的交接处，包括所有窗体、报表、打印机和扫描仪等硬件的接口，以及与其他系统的接口。要寻找和定义边界类，可以检查用例模型，每个参与者和用例交互至少要有一个边界类，边界类使参与者能与系统交互。边界类是一种用于对系统外部环境与其内部运作之间的交互进行建模的类。常见的边界类有窗口、通信协议、打印机接口、传感器和终端等。实际上，在系统设计时，产生的报表都可以作为边界类来处理。

边界类用于系统接口与系统外部进行交互，边界对象将系统与其外部环境的变更（例如，与其他系统的接口的变更、用户需求的变更等）分隔开，使这些变更不会对系统的其他部分造成影响。通常情况下，边界类可以既有属性也有方法。

2、系统测试由若干个不同的测试类型组成，其中（ ）检查系统能力的最高实际限度，即软件在一些超负荷情况下的运行情况；（ ）主要是检查系统的容错能力。

A 强度测试 B 性能测试 C 恢复测试 D 可靠性测试

A 强度测试 B 性能测试 C 恢复测试 D 可靠性测试

参考答案： A C

试题解析：

系统测试是将已经确认的软件、计算机硬件、外设和网络等其他因素结合在一起，进行信息系统的各种组装测试和确认测试，其目的是通过与系统的需求相比较，发现所开发的系统与用户需求不符或矛盾的地方。系统测试是根据系统方案说明书来设计测试例子的，常见的系统测试主要有以下内容：

·恢复测试：恢复测试监测系统的容错能力。检测方法是采用各种方法让系统出现故障，检验系统是否按照要求能从故障中恢复过来，并在约定的时间内开始事务处理，而且不对系统造成任何伤害。如果系统的恢复是自动的（由系统自动完成），需要验证重新初始化、检查点、数据恢复等是否正确。如果恢复需要人工干预，就要对恢复的平均时间进行评估并判断它是否在允许的范围内。

·安全性测试：系统的安全性测试是检测系统的安全机制、保密措施是否完善，主要是为了检验系统的防范能力。测试的方法是测试人员模拟非法入侵者，采用各种方法冲破防线。系统安全性设计准则是使非法入侵者所花费的代价比进入系统后所得到的好处要大，此时非法入侵已无利可图。

·强度测试：是对系统在异常情况下的承受能力的测试，是检查系统在极限状态下运行时，性能下降的幅度是否在允许的范围内。因此，强度测试要求系统在非正常数量、频率或容量的情况下运行。强度测试主要是为了发现在有效的输入数据中可能引起不稳定或不正确的数据组合。例如，运行使系统处理超过设计能力的最大允许值的测试例子；使系统传输超过设计最大能力的数据，包括内存的写入和读出等。

·性能测试：检查系统是否满足系统设计方案说明书对性能的要求。性能测试覆盖了软件测试的各阶段，而不是等到系统的各部分所有都组装之后，才确定系统的真正性能。通常与强度测试结合起来进行，并同时软件、硬件进行测试。软件方面主要从响应时间、处理速度、吞吐量、处理精度等方面来检测。

·可靠性测试：通常使用以下两个指标来衡量系统的可靠性：平均失效间隔时间 MTBF（mean time between failures）是否超过了规定的时限，因故障而停机时间 MTTR（mean time to repairs）在一年中不应超过多少时间。

·安装测试：在安装软件系统时，会有多种选择。安装测试就是为了检测在安装过程中是否有误、是否容易操作等。主要监测系统的每一个部分是否齐全，硬件的配置是否合理，安装中需要产生的文件和数据库是否已产生，其内容是否正确等。

3、（ ）的目的是检查模块之间，以及模块和已集成的软件之间的接口关系，并验证已集成的软件是否符合设计要求。其测试的技术依据是（ ）。

A 单元测试 **B 集成测试** C 系统测试 D 回归测试

A 软件详细设计说明书 B 技术开发合同

C 软件概要设计文档 D 软件配置文档

参考答案： B C

试题解析：

单元测试也称为模块测试，测试的对象是可独立编译或汇编的程序模块、软件构件或 OO 软件中的类（统称为模块），其目的是检查每个模块能否正确地实现设计说明中的功能、性能、接口和其他设计约束等条件，发现模块内可能存在的各种差错。单元测试的技术依据是软件详细设计说明书。

集成测试的目的是检查模块之间，以及模块和已集成的软件之间的接口关系，并验证已集成的软件是否符合设计要求。集成测试的技术依据是软件概要设计文档。

系统测试的对象是完整的、集成的计算机系统，系统测试的目的是在真实系统工作环境下，验证完整的软件配置项能否和系统正确连接，并满足系统/子系统设计文档和软件开发合同规定的要求。系统测试的技术依据是用户需求或开发合同，除应满足一般测试的准入条件外，在进行系统测试前，还应确认被测系统的所有配置项已通过测试，对需要固化运行的软件还应提供固件。

回归测试的目的是测试软件变更之后，变更部分的正确性和对变更需求的符合性，以及软件原有的、正确的功能、性能和其他规定的要求的不损害性。